FCEFyN
FACULTAD DE CIENCIAS EXACTAS, FÍSICAS Y NATURALES

Redes de Computadoras

Trabajo Práctico

N.º 4(Teórico)

Protocolos de Capa de Transporte

Grupo: NetRunners
Docente: FACUNDO NICOLAS OLIVA CUNEO

Integrantes

Bastida, Lucas Ramiro	DNI 39444511
Contessi, Ruben Andres	DNI 33315235
Garay, Ignacio Hernan	DNI 44191925
Giraudo, Juan Pablo	DNI 34970089
Peñaloza, Gonzalo Adrian	DNI 40441355
Quinteros del Castillo, Tomás Agustín	DNI 46169739
Vasconsellos Blason, Benjamin Alejandro	DNI 45642879

16 de octubre de 2026

20.1 **¿Qué elementos de direccionamiento son necesarios para identificar al usuario TS destino?**

Para identificar al usuario TS destino se necesita:

(a) Identificación del usuario.
(b) Identificación de la entidad de transporte.
(c) Dirección de la estación.
(d) Número de la red.

20.2 **¿Cuáles son las cuatro estrategias que puede utilizar un usuario TS emisor para encontrar la dirección de otro usuario TS?**

(a) El usuario TS emisor ya conoce la dirección del usuario destino.
(b) Utilizar direcciones conocidas para servicios comunes.
(c) Utilizar un servidor de nombres para obtener la dirección correspondiente al usuario destino.
(d) Crear el proceso destino cuando se realiza la solicitud, obteniendo su dirección durante este proceso.

20.3 **Explique el uso de la multiplexación en el contexto de un protocolo de transporte.**

La **multiplexación** permite que el protocolo de transporte utilice de manera más eficiente las conexiones de red. La idea es que varias conexiones de transporte puedan compartir una misma conexión de red, o que una conexión de transporte pueda utilizar varias conexiones de red.

Se pueden distinguir dos casos:

- **Multiplexación hacia arriba (upward multiplexing):** varias conexiones de transporte utilizan una única conexión de red. Esto permite reducir la cantidad de conexiones de red necesarias, compartiendo una misma conexión entre diferentes conexiones de transporte.
- **Multiplexación hacia abajo (downward multiplexing):** una única conexión de transporte utiliza varias conexiones de red. Los datos pueden distribuirse entre estas conexiones, lo que permite aprovecharlas simultáneamente y puede mejorar el rendimiento.

En resumen la multiplexación permite aprovechar mejor los recursos de la red, ya sea haciendo que varias conexiones de transporte compartan una misma conexión de red, o utilizando varias conexiones de red para una misma conexión de transporte.

20.4 **Describa brevemente el esquema de créditos utilizado por TCP para el control de flujo.**

TCP utiliza un **esquema de créditos** para realizar el control de flujo y evitar que el emisor envíe más datos de los que el receptor puede almacenar.

Cada octeto de datos posee un número de secuencia, y los segmentos TCP incluyen un número de secuencia, un número de confirmación (ACK) y un tamaño de ventana. El receptor utiliza la ventana para indicar al emisor cuántos octetos adicionales está preparado para recibir.

Por ejemplo, si un ACK contiene $AN = i$ y una ventana $W = j$, significa que todos los octetos anteriores a i fueron recibidos correctamente, que se espera recibir el octeto i y que el emisor tiene permiso para enviar hasta j octetos adicionales.

De esta manera, el receptor puede informar dinámicamente cuánto espacio disponible tiene en su buffer. Si la ventana es pequeña, el emisor debe reducir la cantidad de datos que envía; si aumenta, puede enviar más datos.

Una característica importante del esquema de créditos de TCP es que **el control de flujo está desacoplado de las confirmaciones**. Es decir, un segmento puede confirmar datos recibidos sin necesariamente otorgar nuevos créditos, o puede otorgar créditos sin confirmar nuevos datos.

20.5 **¿Cuál es la diferencia principal entre el esquema de créditos de TCP y el esquema de control de flujo de ventana deslizante utilizada por muchos otros protocolos, como por ejemplo HDLC?**

En el esquema de ventana deslizante, la confirmación o ACK de la llegada de datos habilita al emisor inmediatamente a enviar más datos. En cambio, en el sistema de créditos se puede dar el ACK de que los datos llegaron, pero la habilitación para enviar más datos es independiente. Por lo tanto, el emisor puede recibir la confirmación de la llegada de los datos, pero no tener permitido enviar más hasta recibir créditos adicionales para continuar la comunicación.

20.6 **Explique los mecanismos de diálogo en dos y tres pasos.**

En un diálogo en dos pasos entre dos interlocutores A y B, A inicia la solicitud de conexión con un mensaje SYN i. B responderá con un SYN j y confirmará la recepción del SYN de A con un ACK (i + 1). En el diálogo de tres pasos, A enviará un SYN i, B lo recibe y responde con SYN j y ACK (i + 1). Cuando A recibe el SYN de B, responde con ACK (j + 1). Los valores i y j serán números de secuencia elegidos de forma independiente por cada uno de los interlocutores.

20.7 **¿Cuál es el beneficio del mecanismo de diálogo en tres pasos?**

El establecimiento de una conexión debe tener en cuenta la falta de fiabilidad del servicio de red. Si A envía un SYN a B, pueden ocurrir dos problemas:

(a) Que se pierda el SYN de A o

(b) Que se pierda la respuesta de B

Esto se puede resolver retransmitiendo SYN. Sin embargo puede generar duplicados. Por ejemplo, si la respuesta de B solamente estaba retrasada, A enviará otra solicitud que B ya recibió.

También pueden quedar segmentos de datos viejos. Si una nueva conexión utiliza los mismos números de secuencia que una anterior, estos segmentos podrían confundirse. Cada conexión debe comenzar con un número de secuencia diferente al anterior.

También un SYN viejo puede llegar después del cierre de una conexión y ser interpretado como una solicitud nueva y se conectan aunque estén esperando números de secuencia distintos. Esto se resuelve con el dialogo en tres pasos:

(a) A envía un SYN con número de secuencia inicial.

(b) B confirma el SYN de A y envía su propio SYN con número de secuencia inicial propio.

(c) A confirma el SYN de B.

El beneficio es que cada extremo confirma explícitamente el SYN y número de secuencia inicial del otro antes de considerarse establecida la conexión. De esta manera evita conexiones incorrectas por la solicitud vieja o duplicada y se asegura sobre los números de secuencia que van a utilizar

20.8 **Defina las características de urgencia y forzado de TCP.**

TCP almacena temporalmente los datos tanto al enviarlos como al recibirlos. Normalmente, decide cuándo formar un segmento para transmitirlo y cuándo entregar los datos recibidos a la aplicación. Además, ofrece dos servicios útiles para etiquetar los datos: forzado y urgente.

- **Flujo de datos forzados:** permite solicitar que TCP envíe los datos pendientes sin esperar a acumular más. En el extremo receptor, indica que los datos deben entregarse a la aplicación sin seguir reteniéndose a la espera de más datos. Esto resulta útil cuando se termina una parte lógica de la información y se necesita que el receptor pueda procesarla.
- **Señalización de datos urgentes:** permite avisar a la aplicación receptora que hay datos que requieren atención especial. TCP utiliza un puntero de urgente para señalar el final del bloque de datos urgentes. Estos se transmiten dentro del flujo de datos normal. La aplicación receptora es responsable de decidir qué acción realizar.

En la cabecera TCP se encuentran los indicadores correspondientes de 6 bits:

- **PSH:** señala la función de forzado.
- **URG:** indica que el campo de puntero urgente es válido.

La diferencia es que el forzado busca evitar la espera por acumulación de datos, mientras que la urgencia señala que una parte de los datos requiere atención especial.

20.9 **¿Qué es una opción en los criterios de implementación de TCP?**

El estándar de TCP (RFC 793) especifica el formato de los segmentos y las reglas del protocolo, pero deja libertad en algunos aspectos de implementación: aunque dos TCP elijan opciones distintas, el protocolo sigue funcionando igual, pero el rendimiento puede variar según qué política haya elegido cada uno. Podemos nombrar cinco áreas (las más importantes):

- **Política de envío:** cuándo el emisor arma un segmento con los datos que tiene acumulados (enviar apenas tenga algo disponible, o esperar a juntar más datos antes de armar el segmento).
- **Política de entrega:** lo mismo que en la de envío, pero del lado del receptor (entregar los datos a la aplicación apenas llega cada segmento en orden, o acumular varios segmentos antes de entregarlos).
- **Política de aceptación:** qué hacer si llegan segmentos desordenados (aceptar lo que llega en secuencia y descartar el resto, o aceptar y guardar cualquier segmento que caiga dentro de la ventana de recepción aunque esté desordenado).
- **Política de retransmisión:** qué hacer cuando vence el temporizador de un segmento no confirmado.
- **Política de confirmación:** cuando llega un segmento en orden, se confirma al instante de la llegada o se espera a pisar la confirmación sobre el próximo.

20.10 **¿Cómo puede utilizarse TCP para tratar la congestión de red o de interconexión de red?**

TCP controla la congestión de forma indirecta, ya que no puede controlar directamente los elementos de la red. Detecta posibles situaciones de congestión a partir de la **pérdida de paquetes, ACKs duplicados o aumentos en los tiempos de espera**. Para regular la cantidad de datos que transmite, utiliza principalmente los siguientes mecanismos:

- **Ventana de congestión** ($CWND$): limita la cantidad de datos que pueden encontrarse en tránsito. La cantidad efectiva de datos enviados está limitada por el mínimo entre $CWND$ y la ventana anunciada por el receptor ($RWIN$).
- **Arranque lento (Slow Start)**: al comenzar una conexión o después de una congestión importante, TCP inicia con una ventana pequeña y aumenta rápidamente $CWND$, buscando determinar progresivamente la capacidad disponible de la red.
- **Evitación de congestión (Congestion Avoidance)**: cuando $CWND$ alcanza el umbral *ssthresh*, su crecimiento se vuelve más lento y aproximadamente lineal, evitando sobrecargar la red.
- **Retransmisión rápida y recuperación rápida (Fast Retransmit / Fast Recovery)**: cuando TCP recibe varios ACKs duplicados, interpreta que probablemente se perdió un segmento. Lo retransmite sin esperar a que expire el temporizador y reduce la ventana de congestión para disminuir la carga sobre la red.
- **Temporizador de retransmisión (RTO)**: TCP adapta el tiempo de espera para las retransmisiones en función de las estimaciones del RTT. Si aumenta el retardo de la red, el RTO se ajusta para evitar retransmisiones prematuras que podrían aumentar aún más la congestión.

En conjunto, estos mecanismos permiten que TCP **adapte dinámicamente la velocidad de transmisión a las condiciones de la red**, reduciendo el tráfico cuando detecta congestión y aumentando progresivamente la transmisión cuando la red parece tener capacidad disponible.

20.11 **¿Qué proporciona UDP que no ofrezca IP?**

El Protocolo de Datagramas de Usuario UDP (User Datagram Protocol) es un protocolo de la capa de transporte orientado a no conexión que ofrece un servicio de entrega de mensajes no confiable y de mínimo esfuerzo sobre la capa de red.

Específicamente, UDP añade sobre el servicio básico de IP dos funciones fundamentales de la capa de transporte:

- **Puertos de origen y destino:** Permite la multiplexación y desmultiplexación del tráfico, entregando los datos a un proceso o aplicación específica dentro del equipo de destino, a diferencia de IP que solo entrega el paquete de equipo a equipo.
- **Suma de comprobación (Checksum):** Ofrece una verificación de errores de extremo a extremo que protege tanto la cabecera UDP como la carga útil de datos y una pseudo-cabecera IP, asegurando que el mensaje no haya sufrido corrupción en el trayecto. En IPv6, este mecanismo es obligatorio.

Actividad 2: Responder por lo menos 11 preguntas (a su elección) de la sección EJERCICIOS, pag 719, del Capítulo 20. Protocolos de transporte, Stallings - Comunicaciones y Redes de Computadores 7ed.

1 Consignas

20.1 Es una práctica común en la mayoría de los protocolos de transporte (en realidad, en la mayoría de los protocolos de todas las capas) que los datos y la señalización de control se multiplexen sobre el mismo canal lógico en cada conexión por usuario. Una alternativa consiste en establecer una única conexión de control de transporte entre cada par de entidades de transporte que se comuniquen. Esta conexión se usaría para transmitir las señales de control de todas las conexiones de los usuarios de transporte entre las dos entidades. Discuta las implicaciones de esta estrategia.

Pensamos que juntar toda la señalización de control en una única conexión de transporte entre cada par de entidades, en lugar de llevarla mezclada con los datos de cada conexión de usuario como se hace normalmente, trae alguna ventaja pero también complica bastante las cosas.

Lo positivo es que la entidad de transporte dejaría de manejar un canal de control por cada conexión de usuario y pasaría a tener uno solo por cada par de entidades que se comunican. Eso simplifica en parte la cantidad de recursos que hay que reservar para señalización y evita duplicar mecanismos de control en cada conexión que se abre.

El problema es que esa única conexión se transforma en un cuello de botella y en un punto único de falla: si se pierde un mensaje de control ahí o si el canal se congestiona, puede afectar a todas las conexiones de usuario que dependen de él para coordinarse, aunque sus datos viajen por canales totalmente distintos. Además, cada mensaje de control tendría que indicar explícitamente a qué conexión de usuario corresponde, porque ya no existe una asociación directa entre canal y conexión como pasa cuando todo va junto. Eso suma una capa de identificación que antes no hacía falta.

También se pierde algo de aislamiento entre conexiones. Si una conexión de usuario genera mucha señalización, por ejemplo porque está retransmitiendo seguido, podría demorar el control de las demás conexiones que comparten ese mismo canal, generando una interferencia cruzada que no existiría si cada una llevara su propia señalización por separado.

En resumen, nos parece que esta estrategia no elimina la complejidad sino que la traslada: ahorra en cantidad de conexiones de control pero suma trabajo de identificación de mensajes y crea una dependencia compartida entre conexiones que en el esquema habitual son completamente independientes entre sí.

20.2 La discusión sobre control de flujo con un servicio de red fiable, referido como mecanismo de contrapresión, utiliza un protocolo de control de flujo de una capa inferior. Discuta las desventajas de esta estrategia.

La **contrapresión** delega el control de flujo de la capa de transporte a la capa de red rechazando paquetes cuando el *buffer* se satura. Sus principales desventajas son:

(a) **Falta de granularidad (multiplexación):** Al compartir varias conexiones de transporte un mismo circuito virtual de red, la contrapresión frena todo el circuito, afectando injustamente a las conexiones que no están saturadas.

(b) **Mecanismo impreciso:** Es tosco comparado con las ventanas deslizantes; no permite indicar la cantidad exacta de espacio libre disponible en el *buffer*.

(c) **Propagación de la congestión:** Acumula paquetes en los nodos intermedios de la red, generando un efecto dominó que bloquea otras zonas no relacionadas.

(d) **Incompatibilidad:** No se puede aplicar en redes no orientadas a conexión o sin control de flujo interno (como redes datagrama o IP).

20.5 Con un servicio de red fiable y ordenado, ¿son estrictamente necesarios los números de secuencia de los segmentos? ¿Qué capacidad se pierde sin ellos?

No, los números de secuencia de los segmentos no son estrictamente necesarios para garantizar la entrega en orden ni la integridad de los datos ya que el servicio de red subyacente asume la responsabilidad de entregar los paquetes de forma fiable y en la secuencia correcta.

Sin embargo, al prescindir de los números de secuencia se pierde la capacidad de implementar mecanismos avanzados de control de flujo en la capa de transporte como la asignación de créditos basada en octetos o segmentos específicos, el reconocimiento explicito o diferenciado de unidades de datos (ACKs individuales/acumulativos), y la capacidad de detectar o recuperarse ante incidencias no previstas por la red, como reinicios del servicio de red, pérdida de mensajes de control o saturación de buffers en los extremos

20.6 Considere un servicio de red orientado a conexión que sufre un reinicio. ¿Cómo podría ser tratado por un protocolo de transporte que suponga que el servicio de red es fiable excepto en el caso de un reinicio?

Cuando un servicio de red orientado a conexión sufre un **reinicio** (*reset*), la capa de red notifica la anomalía a las entidades de transporte en ambos extremos. Dado que los paquetes en tránsito no confirmados pueden haberse perdido o descartado, el protocolo de transporte responde de la siguiente manera:

(a) **Retransmisión de paquetes pendientes:** La entidad emisora asume que todos los segmentos transmitidos sin confirmación previa al reinicio se han perdido y procede a su retransmisión.

(b) **Resincronización de estado:** Ambos extremos intercambian mensajes de control para reordenar los números de secuencia y restablecer los parámetros de la ventana deslizante/créditos.

(c) **Notificación al usuario del servicio (capa superior):** Si la resincronización falla o no es posible garantizar la entrega limpia de datos, la capa de transporte aborta la conexión e informa al nivel superior (aplicación/sesión) para que ejecute un mecanismo de recuperación (por ejemplo, mediante puntos de control o *checkpoints*).

20.7 La discusión de la política de retransmisión hizo referencia a tres problemas asociados con el cálculo dinámico del valor del temporizador. ¿Qué modificaciones sobre la política ayudarían a aliviar estos problemas?

El libro plantea que no se puede confiar del todo en un temporizador calculado dinámicamente a partir del promedio de los retardos observados, por tres razones puntuales: que la entidad remota puede no confirmar cada segmento de inmediato porque está usando confirmaciones acumuladas, que si un segmento fue retransmitido el emisor no tiene forma de saber si el ACK que llegó corresponde a la transmisión original o a la retransmisión, y que las condiciones de la red pueden cambiar de un momento a otro.

Para el primer problema, el de las confirmaciones acumuladas que retrasan o agrupan los ACK, ayuda no depender de una sola muestra de RTT para ajustar el temporizador, sino promediar varias observaciones dándole más peso a las últimas, tal como hace el promediado exponencial en vez de un promedio simple.

Para el segundo, el de no saber si un ACK corresponde a la transmisión original o a una retransmisión, la modificación más directa es el algoritmo de Karn: directamente se descarta la muestra de RTT de cualquier segmento que haya sido retransmitido y no se usa para actualizar la estimación. En su lugar se aplica un valor de temporizador por decaimiento exponencial, que va creciendo con cada retransmisión sucesiva del mismo

segmento, hasta que llega la confirmación de uno que no tuvo que retransmitirse y recién ahí se retoma el cálculo normal.

Y para el tercero, los cambios bruscos en las condiciones de la red, conviene que la estimación reaccione rápido en lugar de promediar todo por igual. El algoritmo de Jacobson apunta justamente a esto: además de suavizar el RTT (SRTT) estima su variabilidad (SDEV) y la suma al cálculo del RTO, de forma que si el retardo empieza a variar mucho de golpe el temporizador crece enseguida, en vez de tardar varias muestras en adaptarse como pasaría con un promedio simple.

20.8 Considere un protocolo de transporte que usa un servicio de red orientado a conexión. Suponga que ese protocolo de transporte utiliza un esquema de asignación de créditos para el control de flujo y que el protocolo de red usa un esquema de ventana deslizante. ¿Qué relación, si existe, debería haber entre la ventana dinámica del protocolo de transporte y la ventana fija del protocolo de red?

No existe una relación matemática ni un acoplamiento estricto entre ambas ventanas, ya que la ventana fija de red gestiona el flujo entre nodos intermedios mientras que la ventana dinámica de transporte controla el flujo extremo a extremo entre los procesos. Desde el punto de vista del rendimiento, la ventana de la red establece un límite práctico sobre los datos en tránsito; si la ventana de transporte supera a la de red, los segmentos se acumularán en los buffers del emisor o se descartarán por saturación. Por esto, la ventana de transporte no debería exceder la capacidad de transporte de la ventana de red.

20.10 ¿Es posible que se produzca un bloqueo mutuo utilizando un diálogo en dos pasos en lugar de un diálogo en tres pasos? Dé un ejemplo o demuéstrelo en caso contrario.

Sí, es posible. Cuando se utiliza un diálogo en dos pasos (*two-way handshake*) sobre un servicio de red no fiable, pueden producirse situaciones de bloqueo mutuo o asignación indefinida de recursos no sincronizada.

Ejemplos y Demostración

1. Arribo de un segmento SYN obsoleto o retrasado

(a) La estación **A** envía una solicitud de conexión (`SYN i`) a la estación **B**.

(b) El paquete `SYN i` sufre un gran retardo en la red. El temporizador de **A** expira y **A** desiste del intento de conexión.

(c) Tiempo después, el paquete `SYN i` obsoleto llega finalmente a **B**.

(d) En un diálogo en dos pasos, **B** supone que es una solicitud nueva, asigna recursos y responde con un `SYN j`, pasando a estado *establecido*.

(e) **A** recibe el `SYN j` inesperado y lo descarta, ya que no tiene ninguna solicitud activa.

(f) **Resultado: B** queda convencido de que la conexión está abierta y se mantiene esperando datos indefinidamente, bloqueando recursos para un diálogo que nunca ocurrirá.

2. Pérdida del mensaje de confirmación de B

(a) **A** envía un `SYN` a **B**.

(b) **B** recibe la solicitud, asigna recursos, pasa a estado *establecido* y envía su respuesta de aceptación.

(c) La respuesta de **B** se pierde en la red.

(d) **A** nunca recibe la respuesta, su temporizador expira y permanece en estado *cerrado*.

(e) **Resultado:** **B** permanece bloqueado en estado establecido esperando tráfico de **A**, mientras que **A** considera que la conexión jamás se estableció.

Solución: El diálogo en tres pasos (*Three-Way Handshake*)

Para prevenir el bloqueo mutuo, protocolos como TCP requieren un tercer paso:

(a) **A** → **B:** Envía `SYN i`.

(b) **B** → **A:** Responde con `SYN j, ACK i+1` y pasa a un estado intermedio (`SYN RECEIVED`), **sin** declarar la conexión establecida.

(c) **A** → **B:** Envía `ACK j+1`. Solo al recibir esta confirmación, **B** pasa al estado *establecido*.

Si llega un `SYN` obsoleto, **A** responderá con un segmento de reinicio (`RST`), haciendo que **B** abandone la espera y libere inmediatamente sus recursos.

20.11 A continuación, se enumeran cuatro estrategias que se pueden utilizar para proporcionar a un usuario de transporte las direcciones de un usuario de transporte destino. Para cada una, describa una analogía con el usuario del servicio de correo postal:

a) **Conocer la dirección de antemano.**
Es como cuando ya tenemos anotada en nuestra agenda la dirección postal de alguien porque nos la dieron en algún momento anterior: no hace falta preguntarle a nadie ni averiguar nada, directamente ponemos esa dirección en el sobre y lo despachamos.

b) **Hacer uso de una dirección «bien conocida».**
Se parece a mandarle una carta a una dependencia pública fija, como la municipalidad de una ciudad o la oficina de correos central: no importa quién esté trabajando ahí en ese momento, todo el mundo sabe que esa es la dirección a la que hay que escribirle para ese trámite en particular.

c) **Utilizar un servidor de nombres.**
Es equivalente a llamar a informaciones, o buscar en una guía, para pedir la dirección de una persona o empresa de la que sólo tenemos el nombre: un tercero de confianza nos devuelve el dato que nos falta para poder mandar la carta.

d) **El destinatario se genera al realizar la solicitud.**
Sería como mandarle una carta al sector de «Atención al Cliente» de una empresa: el destinatario puntual que termina recibiéndola y haciéndose cargo del caso recién queda definido cuando la carta llega y se abre, no antes de eso.

20.12 En un esquema de créditos para control de flujo como el de TCP, ¿qué provisión de créditos se podría hacer para la asignación de créditos que se pierdan o se desordenen durante la transmisión?

Para hacer frente a la pérdida o desorden de las asignaciones de crédito, la entidad de transporte puede implementar un temporizador de retransmisión de créditos o enviar actualizaciones de ventana periódicas o acumulativas. Si una asignación de créditos se pierde o llega fuera de orden, la recepción de un segmento subsiguiente con un número de confirmación o valor de ventana actualizado reasigna el estado correcto. Asimismo, si el emisor se

queda sin créditos por la pérdida de un mensaje, puede enviar un segmento de sondeo para forzar al receptor a reemitir su asignación de créditos actual y evitar un bloqueo mutuo (deadlock).

20.13 ¿Qué ocurre en la Figura 20.3 si llega un SYN mientras el usuario solicitado está en el estado CLOSED? ¿Hay alguna forma de llamar la atención del usuario cuando no esté preparado?

En la Figura 20.3, para que la llegada de un SYN derive en una conexión establecida, el usuario TS que lo recibe tiene que estar en el estado LISTEN esperando ese pedido. Si en cambio ese usuario está en CLOSED, es decir, inactivo y sin haber avisado que está disponible, el libro plantea tres caminos posibles para resolverlo.

Uno es que la entidad de transporte directamente rechace la solicitud mandando un segmento RST a quien la envió, cortando el intento ahí mismo. Otro es poner la solicitud en una cola y dejarla esperando hasta que el usuario local decida emitir una orden Open y recién ahí se la atienda. Y el tercero, que es justamente la forma de llamarle la atención al usuario cuando no está preparado, consiste en que la entidad de transporte lo interrumpa para avisarle que hay una solicitud pendiente, en lugar de rechazarla sin más o dejarla esperando sin que nadie se entere. En este último caso ni siquiera hace falta que el usuario haya emitido antes una orden Passive Open: alcanza con que responda a esa interrupción con una orden Accept para aceptar la conexión.

20.14 En la discusión sobre el cierre de la conexión con referencia a la Figura 20.8, se estableció que además de recibir una confirmación de su segmento FIN y enviar una confirmación del segmento FIN recibido, una entidad TCP debe esperar un intervalo de tiempo igual al doble del máximo tiempo de vida esperado de un segmento (el estado TIME WAIT). La recepción de un ACK de su segmento FIN le asegura que todos los segmentos que ha enviado han sido recibidos por el otro extremo. El envío de un ACK del segmento FIN del otro extremo asegura a la otra entidad que todos sus segmentos han sido recibidos. Dé una razón por la que se necesite aún esperar antes de cerrar la conexión.

A pesar de haber recibido la confirmación de su segmento `FIN` y haber confirmado el `FIN` del otro extremo, una entidad TCP debe permanecer en el estado `TIME_WAIT` durante un intervalo de $2 \times$ MSL (el doble del tiempo máximo de vida esperado de un segmento) por dos razones técnicas fundamentales:

(a) **Garantizar la entrega fiable del último `ACK`:** El envío del `ACK` final no asegura que este haya llegado a su destino. Si se pierde en la red, la entidad remota retransmitirá su segmento `FIN`. Mantenerse en `TIME_WAIT` le permite a la entidad local conservar el contexto de la conexión para detectar el `FIN` retransmitido y responder con otro `ACK`, garantizando un cierre ordenado en ambos extremos. Si se cerrara la conexión de inmediato, el `FIN` retransmitido recibiría un segmento de reinicio (`RST`).

(b) **Permitir la disipación de segmentos duplicados u obsoletos:** Asegura que cualquier segmento retrasado perteneciente a la sesión que continúe en tránsito sea descartado por los encaminadores antes de que expire su tiempo de vida. Esto previene que una nueva conexión reutilice el mismo par de conectores (*sockets*) y reciba accidentalmente datos viejos interpretándolos como válidos.

20.15 Normalmente, el campo «ventana» de la cabecera TCP da una asignación de créditos en octetos. Cuando se utiliza la opción de «escalado de ventana», el valor del campo «ventana» se multiplica por 2^F, donde F es el valor de la opción de escalado de ventana. El valor máximo de F que acepta TCP es 14. ¿Por qué se limita esta opción a 14?

La opción se limita a $F = 14$ para evitar el solapamiento y la ambigüedad en los números de secuencia de TCP. Dado que el campo de ventana estándar tiene 16 bits (65.535 octetos como máximo), un factor de escala $F = 14$ permite alcanzar una ventana máxima de $65.535 \times 2^{14} \approx 1$ gigabyte (2^{30} bytes).

La cabecera de TCP define un campo de número de secuencia de 32 bits (2^{32} bytes de espacio total) y una ventana con un factor $F > 14$ superaría los 2^{30} bytes y excedería la mitad de dicho espacio (2^{31} bytes). Esto violaría la regla fundamental de TCP que exige que la ventana no abarque más de la mitad del espacio de números de secuencia, garantizando que el receptor pueda distinguir unívocamente entre segmentos nuevos y antiguos retransmitidos.